

Workflow

Une plateforme de gestion du travail, et une couche d'intelligence artificielle qui classe les tickets, repère les doublons et signale les informations manquantes.

Sommaire

01 Contexte et problématique

02 Architecture microservices

03 La couche d'intelligence artificielle

04 Sécurité, jetons et rôles

O I

Contexte et problématique

Ce qu'un outil de suivi ne fait pas

- Un outil de tickets se remplit plus vite qu'il ne se trie : le type, la priorité, l'équipe et la charge sont saisis à la main — ou pas du tout.
- Le même incident est signalé deux ou trois fois, par des personnes qui ne se lisent pas.
- Un ticket arrive sans version, sans étapes de reproduction, sans capture, et le travail commence par un aller-retour.

Ce que fait la plateforme

Le socle classique

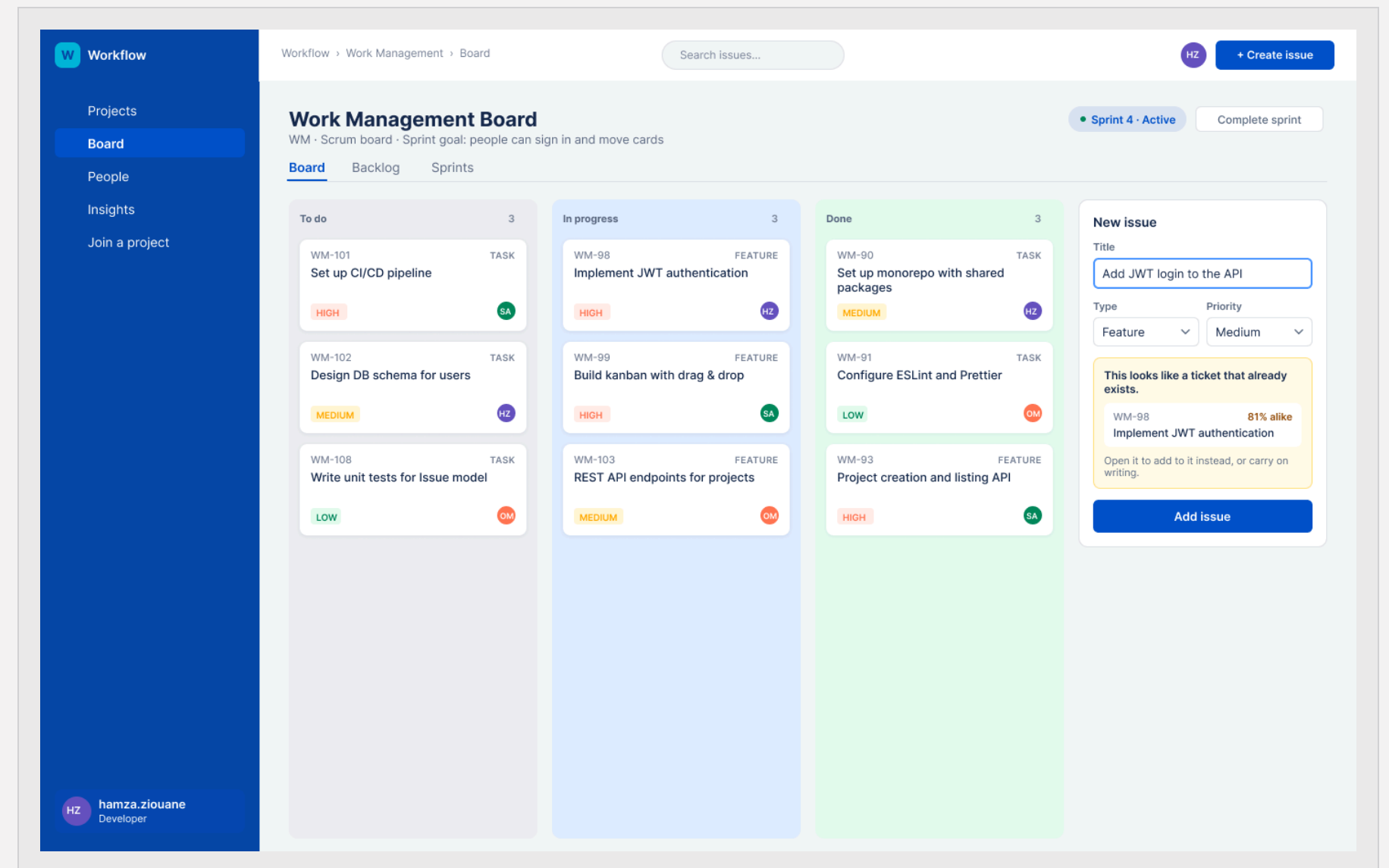
Des projets, des tableaux Kanban et Scrum, des sprints, des tickets, des commentaires, des pièces jointes et l'historique des déplacements de cartes. Le glisser-déposer sur le tableau appelle une seule route ; le backlog n'est pas une table, c'est un sprint vide.

La couche ajoutée

À chaque création de ticket, un modèle propose un type, une priorité, une équipe, un ordre de grandeur d'effort et un sentiment, avec un indice de confiance. Un panneau signale les tickets déjà ouverts qui ressemblent au vôtre, avant l'envoi.

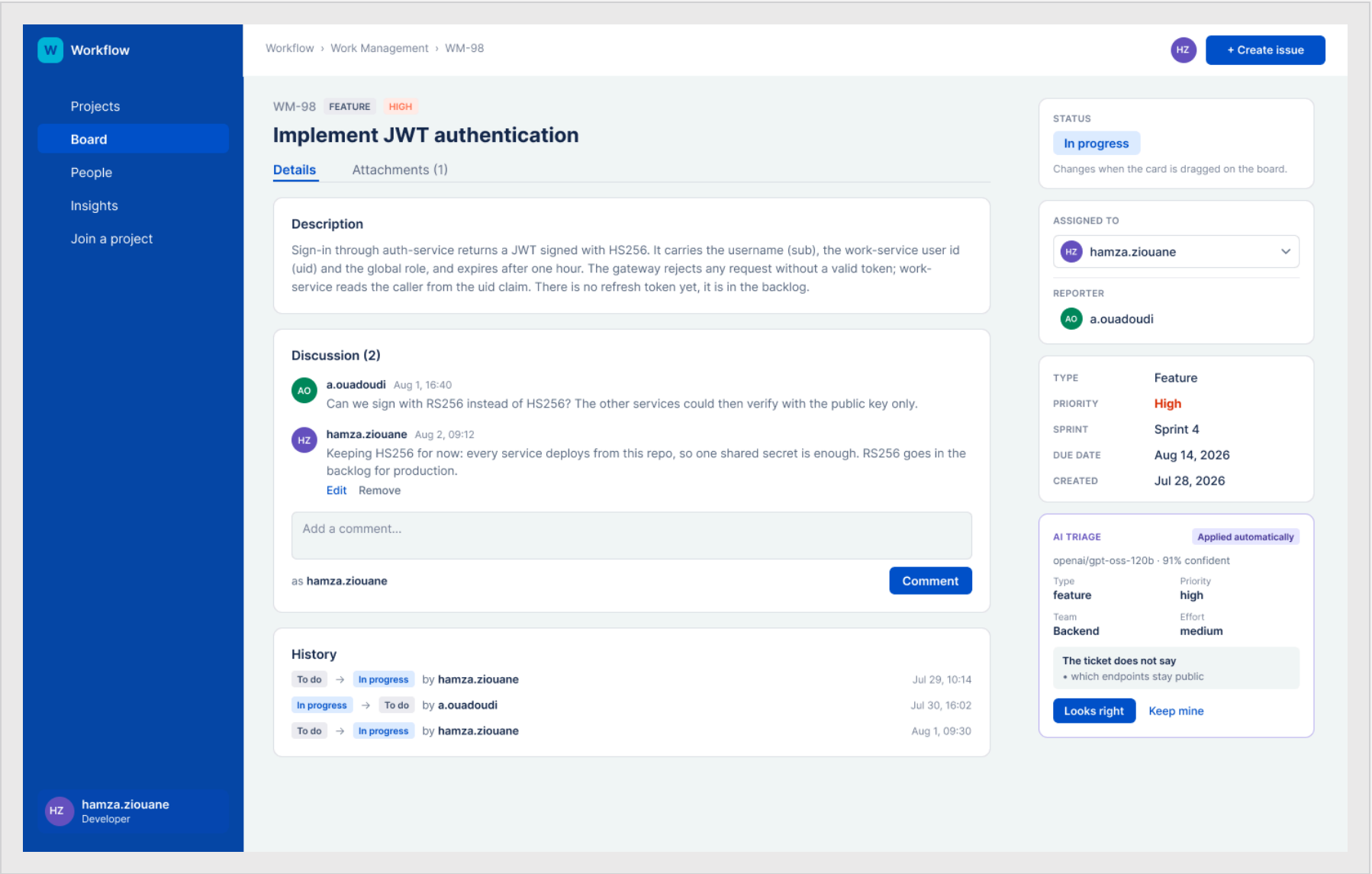
Le tableau

- Les colonnes sont les statuts ; déplacer une carte appelle PATCH `/issues/{id}/status`.
- À droite, la création de ticket annonce un doublon probable avant l'envoi.
- Un seul sprint actif par tableau, garanti par un index unique partiel.



Le ticket

- La suggestion du modèle est une puce : accepter, corriger, ignorer.
- Les valeurs suggérées vivent dans AIClassification ; les valeurs confirmées sur Issue.
- L'historique des déplacements est une table à part, que rien ne peut modifier après coup.



Un jalon, une démo



L'infrastructure n'entre qu'au jalon qui en a besoin : M1 est volontairement un seul service, et la découpe en microservices n'arrive qu'à M2, quand l'authentification apparaît. À la fin de chaque jalon, l'application se démontre.

« L'IA propose, l'humain confirme : les valeurs suggérées vivent à part, les valeurs confirmées sur le ticket. »

— PROJECT.md, principes du projet

02

Architecture microservices

Pourquoi des microservices

La réponse honnête

Le cahier des charges demande de démontrer le motif : passerelle, annuaire de services, messagerie asynchrone, déploiement indépendant. Ce n'est pas un problème de charge — deux utilisateurs tourneraient très bien sur le monolithe que M1 est déjà.

La frontière qui paie vraiment

Une seule découpe est gagnante en soi : la classification. Elle appelle un service externe lent et faillible, et le ticket existe qu'il soit classé ou non. La sortie du chemin de création rend la latence du modèle invisible à celui qui saisit.

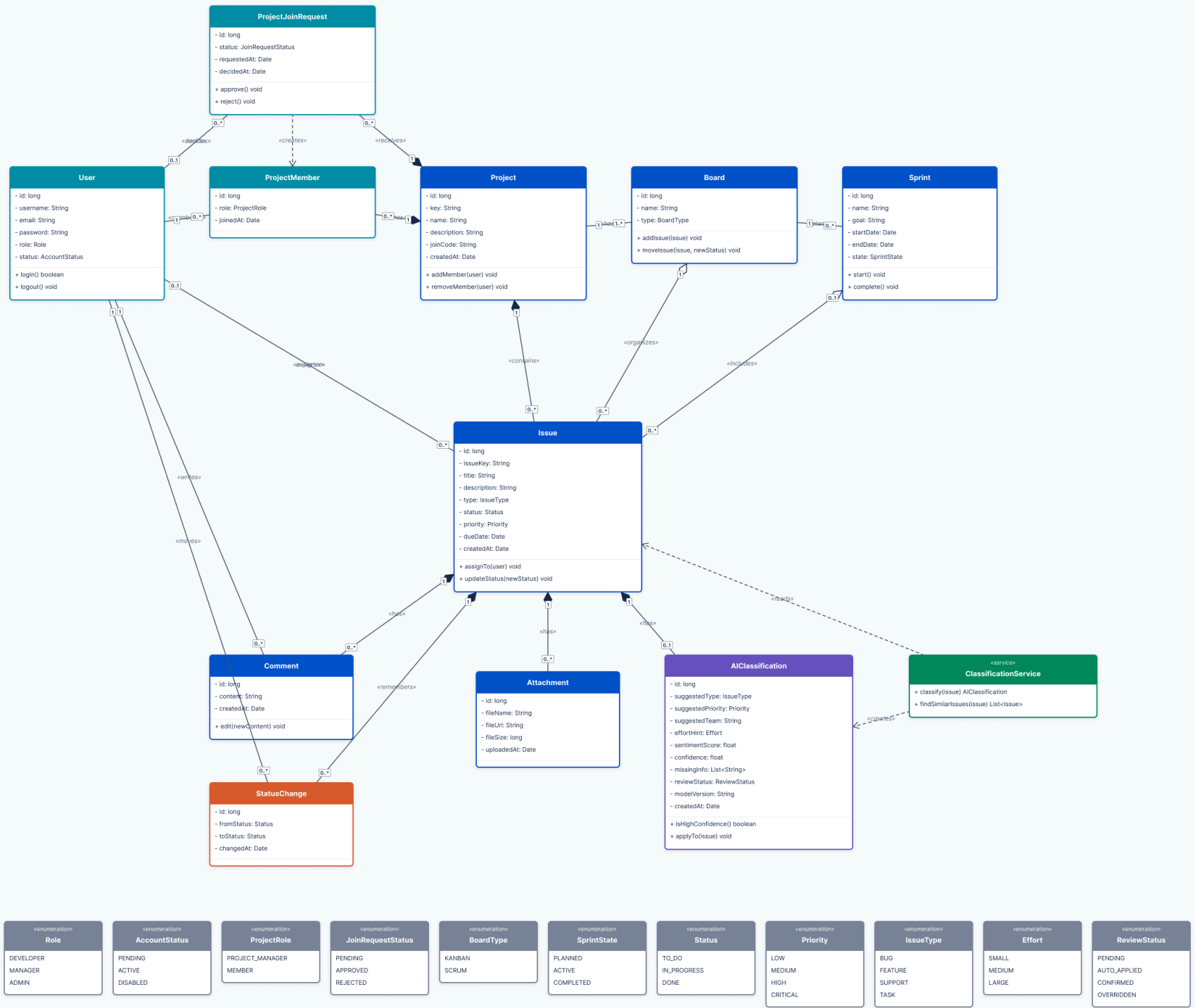
Quatre services

SERVICE	RESPONSABILITÉ	BASE
gateway	Point d'entrée unique : routage, vérification du jeton, CORS, identifiant de corrélation, coupe-circuit	aucune
auth-service	Comptes, connexion, émission du jeton, approbation des inscriptions par un administrateur	authdb
work-service	Projets, tableaux, sprints, tickets, commentaires, historique, classifications, habilitations	workflow
classification-service	Consomme issue.created, appelle Groq, calcule et interroge les vecteurs	vectordb

Autour d'eux : Eureka comme annuaire, RabbitMQ comme file, un seul Postgres 16 avec pgvector. Tous les appels entre services vont dans le même sens, donc il n'y a aucun cycle à casser.

Class Diagram (v2) — UML

Domain model from work-management-class-diagram-v2.mermaid, checked against work-service. Composition = owned lifecycle, aggregation = shared reference, dashed = dependency. Enum references are shown via typed attributes; arrows to enums are omitted for readability.

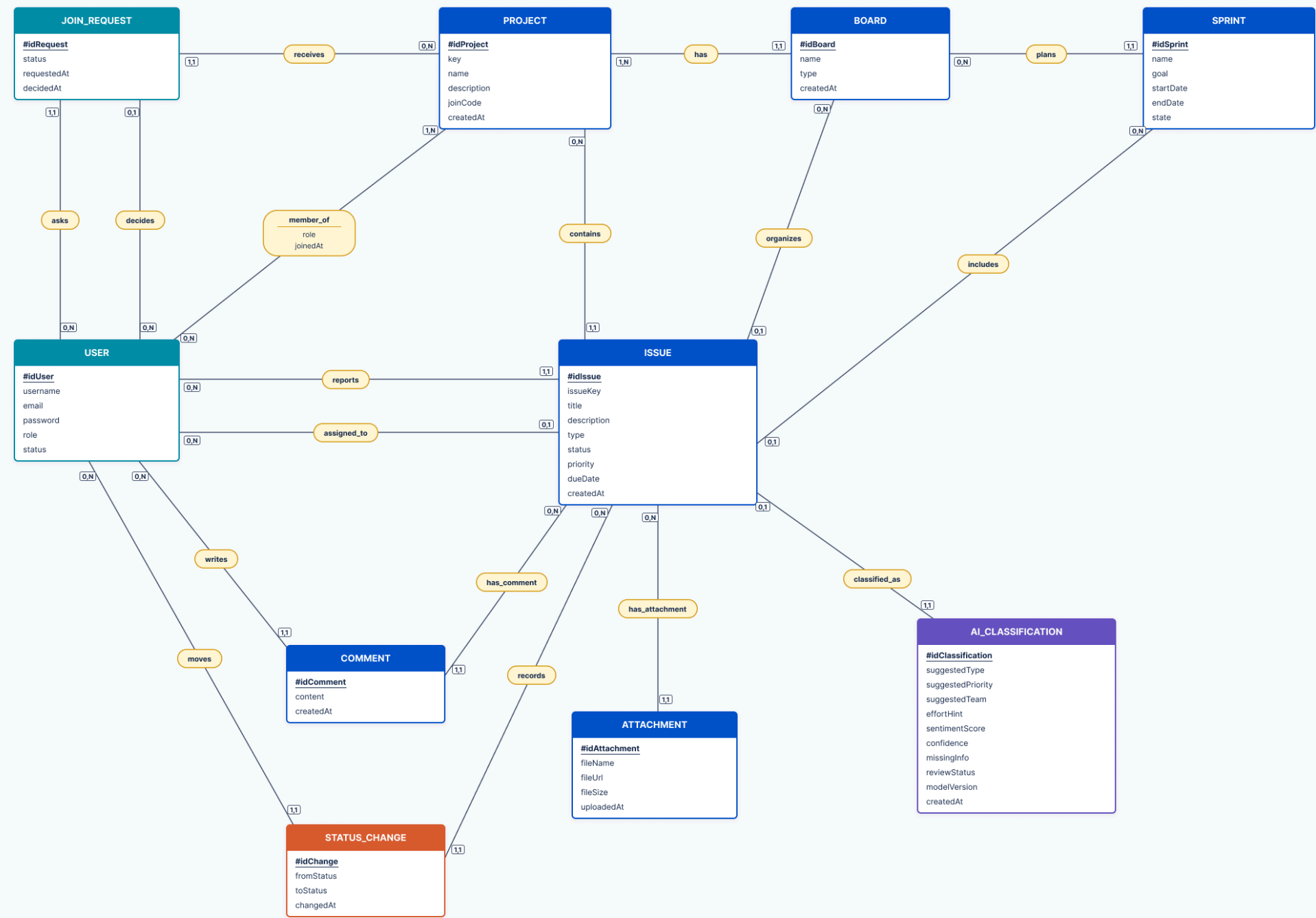


Légende: Work People & access History AI layer Service class

Relationships: Filled diamond = composition (owned lifecycle) · Hollow diamond = aggregation (shared reference) · Plain line = association · Dashed arrow = dependency (transient call)

MCD Merise (v2) — Work management platform

Conceptual data model of the platform, aligned with the code (milestones M1 to M5): project membership, join requests, status history and the AI classification.



Légende: Work: projects, boards, sprints, issues People and access (M5) History (status changes) AI layer (M3) Association Association carrying data

Cardinalités: Lues "pour un X, combien de Y ?" · 0,1 = optional · 1,1 = exactly one · 0,N = zero or many · 1,N = one or many

Domaines: user.role: DEVELOPER | MANAGER | ADMIN · user.status: PENDING | ACTIVE | DISABLED · member_of.role: PROJECT_MANAGER | MEMBER · join_request.status: PENDING | APPROVED | REJECTED · board.type: KANBAN | SCRUM · sprint.state: PLANNED | ACTIVE | COMPLETED · issue.type: BUG | FEATURE | SUPPORT | TASK · issue.status, fromStatus, toStatus: TO_DO | IN_PROGRESS | DONE · priority: LOW | MEDIUM | HIGH | CRITICAL · effortHint: SMALL | MEDIUM | LARGE · reviewStatus: PENDING | AUTO-APPLIED | CONFIRMED | OVERRIDDEN

Notes: USER is one concept stored by two services: auth-service keeps the password hash and the account status in account (authdb), work-service keeps the profile in app_user. Issue similarity is not an association: classification-service computes it from embeddings in issue_vector (vectordb, pgvector), so nothing links ISSUE to ISSUE here.

Trois bases de données

BASE	PROPRIÉTAIRE	CONTENU	JETABLE ?
workflow	work-service	Dix tables métier : projets, tableaux, sprints, tickets, commentaires, pièces jointes, personnes, classifications, membres, demandes	non
authdb	auth-service	Une table, account — les identifiants et l’empreinte du mot de passe	non
vectordb	classification-service	issue_vector — un vecteur de 384 dimensions par ticket	oui, données dérivées

Aucune jointure d’une base à l’autre : account.work_user_id porte l’identifiant du profil sans clé étrangère. Cette contrainte absente est la frontière de service.

Ce qui protège le système

File de rebut

Un message qui échoue trois fois est garé dans workflow.dlx, pas remis en boucle. Un endpoint d'administration le compte, et un autre le rejoue.

Coupe-circuit

Resilience4j sur les routes de la passerelle comme sur l'appel à Groq : un appel suspendu échoue vite au lieu d'épuiser les fils d'exécution.

Identifiant de corrélation

Posé par la passerelle, recopié dans l'en-tête du message : une requête reste un seul grep, même à travers la file.

Verrous et index

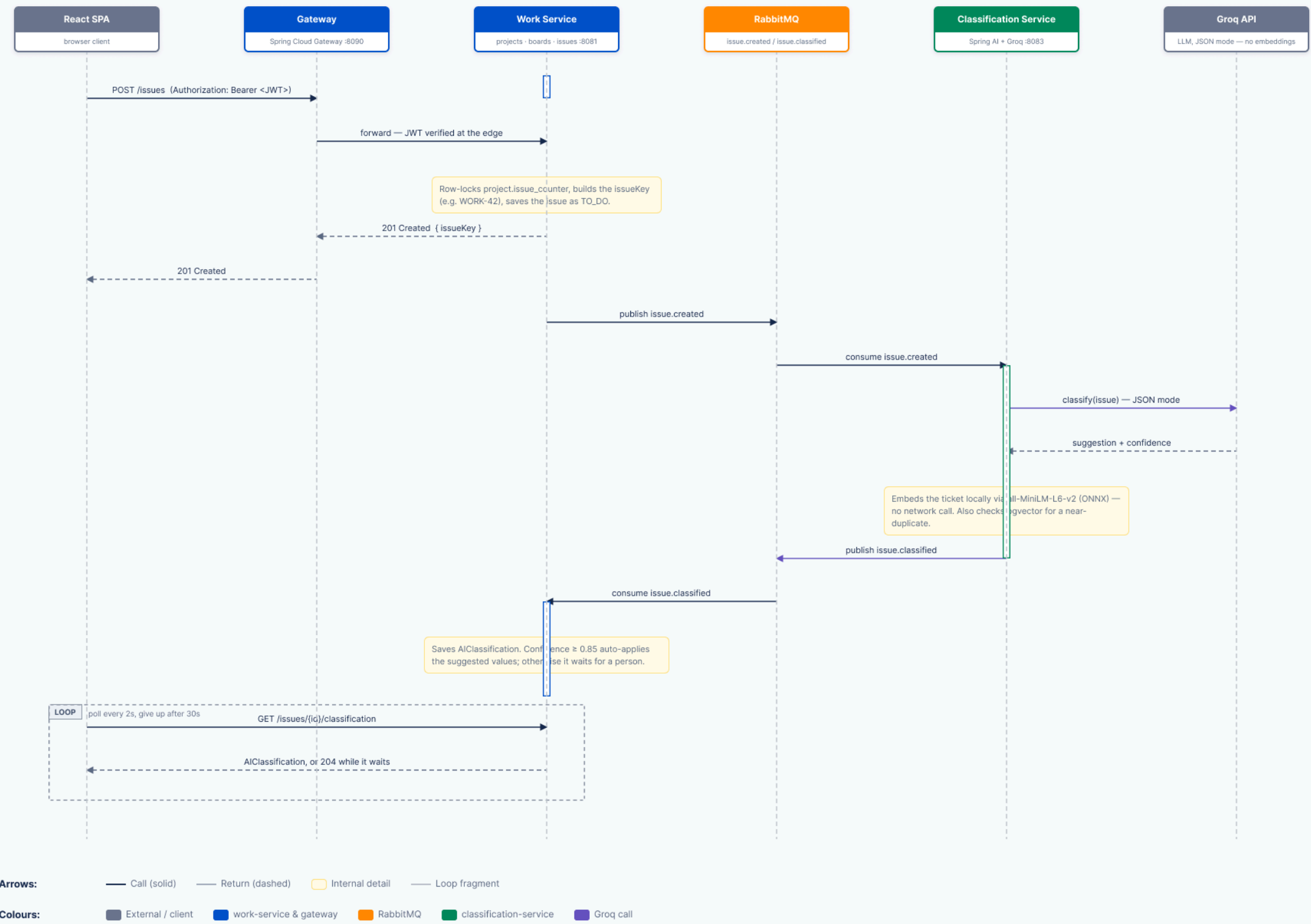
Compteur de tickets sous verrou de ligne, un seul sprint actif par tableau par index unique partiel : la règle est dans la base, pas seulement dans Java.

03

La couche d'intelligence artificielle

Sequence Diagram — Create issue → AI triage (M3, async)

The ticket is saved and returned instantly; classification runs seconds later off a RabbitMQ queue. Solid arrow = call, dashed arrow = return, folded box = note, dashed frame = loop.



Ce que le modèle propose

CHAMP	VALEURS	APPLIQUÉ AU TICKET ?
suggested_type	BUG, FEATURE, SUPPORT, TASK	oui, au-delà du seuil
suggested_priority	LOW, MEDIUM, HIGH, CRITICAL	oui, au-delà du seuil
suggested_team	texte libre — il n’y a pas de table Équipe	non — élément de preuve
effort_hint	SMALL, MEDIUM, LARGE	non — élément de preuve
sentiment_score	–1,0 agacé ... 1,0 satisfait	non — élément de preuve
missing_info	la liste de ce qui manque	affiché, jamais appliqué
confidence	0,0 ... 1,0	c’est lui qui décide

Deux seuils, deux risques

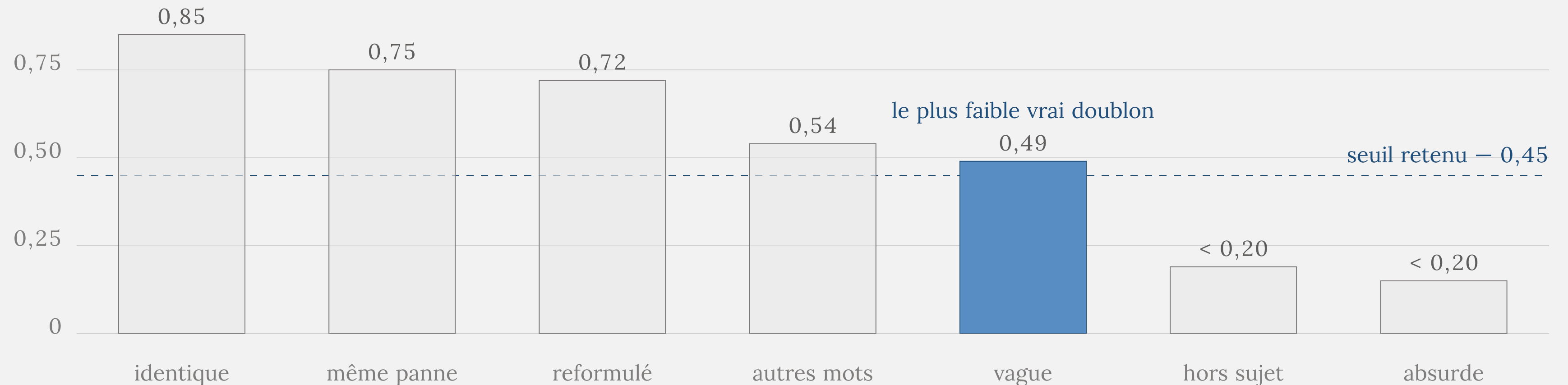
0,85 — appliquer seul

Au-dessus de cette confiance, la suggestion est écrite sur le ticket sans que personne ne la touche, et l'état de revue passe à `AUTO_APPLIED`. Le seuil est haut parce qu'une erreur modifie un ticket sans témoin.

0,45 — proposer un doublon

Le panneau de doublons ne fait qu'offrir une piste. Un faux positif coûte un coup d'œil ; un faux négatif coûte un ticket en double. Le seuil est donc bas — et il a été mesuré, pas choisi.

La mesure derrière 0,45



Sept requêtes contre un même ticket, « Login page crashes with a 500 error ». Les cinq premières décrivent la même panne. Le premier seuil envisagé, 0,75, n'aurait retenu que les formulations quasi identiques — exactement l'échec que la fonctionnalité existe pour empêcher.

Quand le modèle ne répond pas

- Sans clé d'API, un classificateur de repli applique des règles par mots-clés ; chaque ligne porte stub-v1, affiché sur la puce — le projet tourne sur un clone frais.
- 429, 503, délai dépassé, hoquet de base : trois tentatives avec temporisation, puis mise au rebut.
- 401, réponse illisible, message sans identifiant de ticket : au rebut immédiatement — réessayer trois fois ne prouve rien de nouveau.
- Sans default-queue-rejected=false, un message en échec revient en boucle à pleine vitesse : la file de rebut reste vide pendant que le système est bloqué.

04

Sécurité, jetons et rôles

Deux niveaux de rôle

QUESTION	Y RÉPOND	VIT DANS
Que suis-je sur la <i>plateforme</i> ?	DEVELOPER, MANAGER, ADMIN	app_user.role, et la revendication role du jeton
Que suis-je dans <i>ce projet-ci</i> ?	PROJECT_MANAGER, MEMBER	project_member.role, relu à chaque requête

Le rôle de projet n'est délibérément pas une revendication du jeton : il change au moment où un chef de projet ajoute ou retire quelqu'un, et un jeton vit une heure sans moyen de le rappeler. Une donnée d'autorisation qui change appartient à la base, relue sur la requête qui en a besoin.

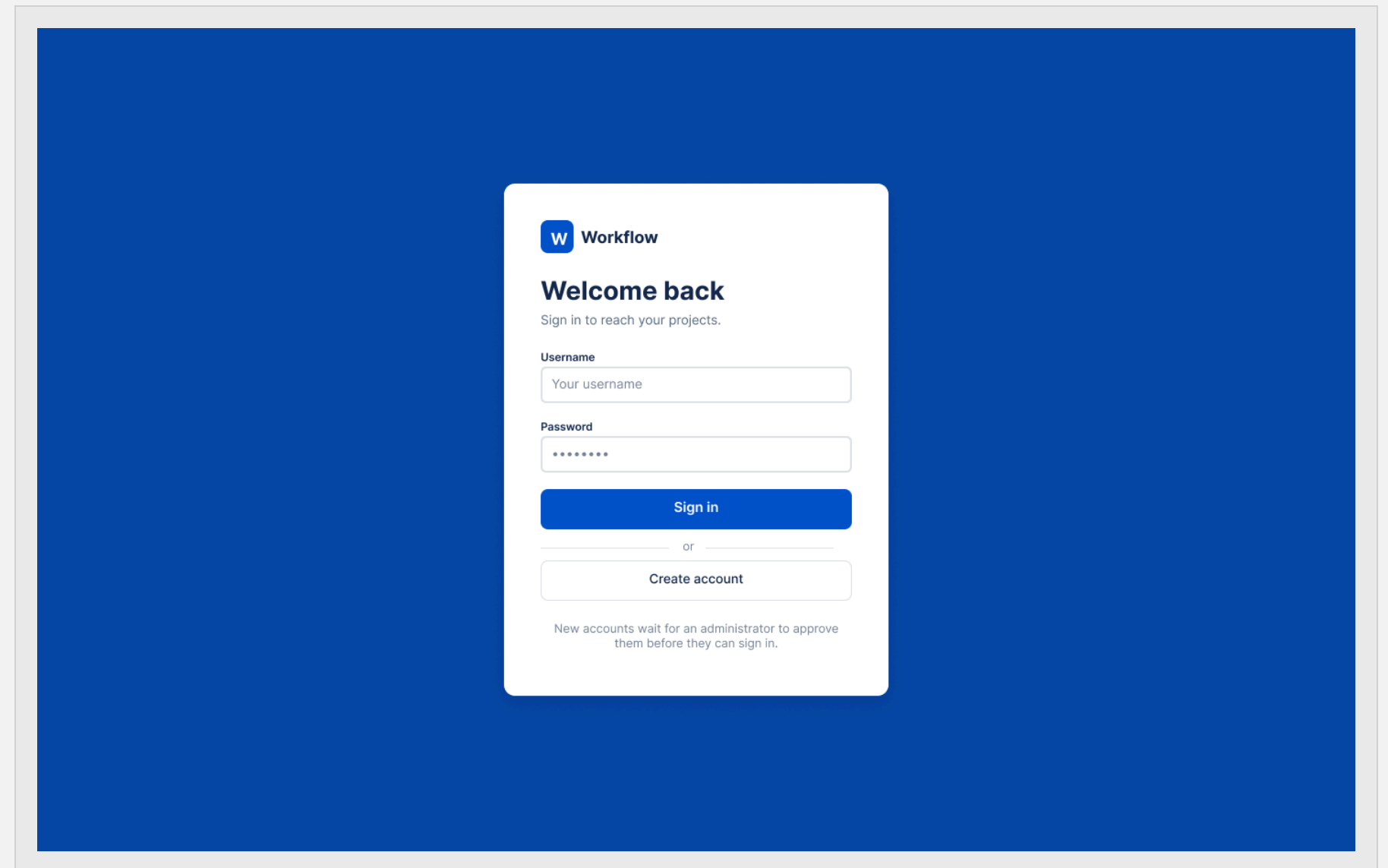
Le jeton est vérifié deux fois

La passerelle n'est pas la frontière
work-service écoute sur 8081 et tout ce qui est sur le réseau peut l'appeler directement : il valide donc le jeton lui-même. La passerelle est l'endroit où une mauvaise requête échoue à bas coût. Démontrable : `curl localhost:8081/projects` sans jeton répond 401.

Les contrôles de ligne sont dans le service
Lire un ticket oblige à le charger avant de savoir à quel projet il appartient — donc avant de savoir si l'appelant a le droit de le voir. ProjectAccess est le point de passage unique : filtrer la seule liste des projets serait décoratif.

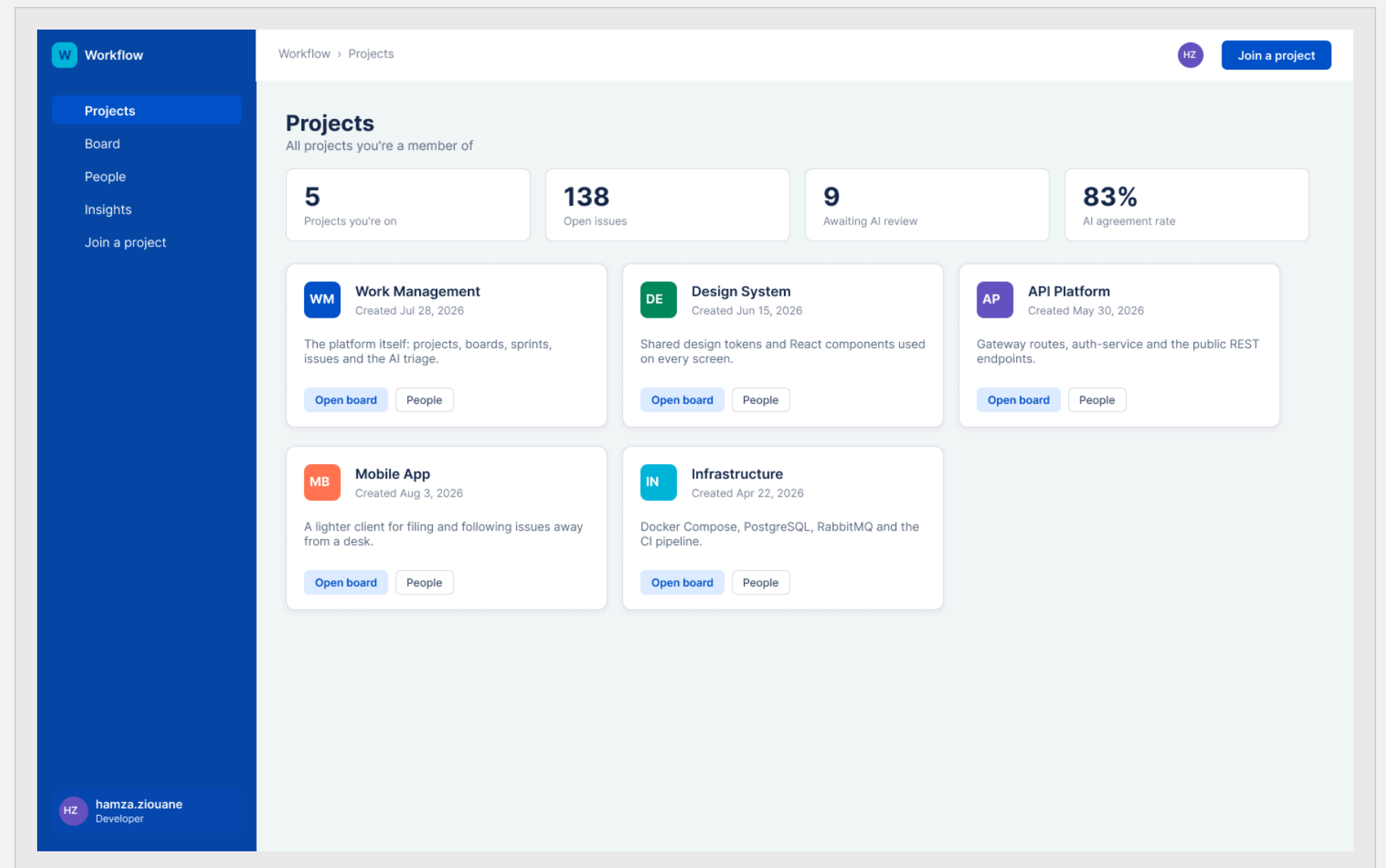
L'inscription est devenue une demande

- La requête d'inscription portait un champ role, sur une route publique : n'importe qui pouvait se déclarer administrateur.
- Tout le monde s'inscrit PENDING, ne peut pas se connecter, et un administrateur dit ensuite ce qu'il est.
- status a remplacé un booléen : « on arrive à vous » et « parlez à votre administrateur » ne sont pas la même phrase.
- Le premier administrateur est semé par une migration — une chaîne d'approbations doit commencer hors d'elle-même.



Rejoindre un projet

- Qui crée un projet en est le premier chef, et gère ses gens lui-même.
- Chaque projet porte un `join_code` secret de douze caractères, que son chef envoie — et qu'il peut faire tourner.
- `project_key` s'affiche sur chaque carte, WM-101 : si c'était aussi la clé d'entrée, quiconque a vu un ticket pourrait demander à entrer.
- Un index unique partiel n'autorise qu'une seule demande en attente par personne et par projet.



Merci

Quatre services, trois bases, six jalons — et une couche d'IA qui propose sans jamais décider seule au-dessous de sa confiance.

Questions